

BIG DATA & STREAMING

Infraestrutura de Ciência de Dados e Big Data com Apache Flink

Arquiteturas Modernas, Integrações Open Source e Inteligência Artificial
em Streaming



Real-Time Processing
Sub-segundo latência



AI Integration
ML_PREDICT & LLMs



Exactly-Once
Garantias de consistência

Professor: Benir Falcão Pistille

Índice

01

Introdução ao Apache Flink

O motor de processamento de streams líder

03

Checkpointing e Tolerância

Exactly-once e recuperação de falhas

05

Camada de IA

ML_PREDICT, LLMs e inferência em tempo real

07

Casos de Uso Práticos

IoT, analytics e detecção de anomalias

02

Arquitetura e Componentes

JobManager, TaskManager e execução distribuída

04

Integrações Open Source

Kafka, Kubernetes e ecossistema

06

Tecnologias Emergentes

Tendências 2025-2026 em Data Infrastructure

08

Ecossistema e Conclusão

Ferramentas complementares e próximos passos

Apache Flink: O Motor de Processamento de Streams

O que é Apache Flink?

Apache Flink é um framework open source de processamento de streams distribuído e stateful para dados em larga escala. Diferente do processamento batch tradicional, o Flink opera continuamente sobre dados à medida que chegam, habilitando **processamento em tempo real com baixa latência**.

Sua arquitetura unificada considera todos os dados como streams, permitindo **um único codebase para batch e streaming**, eliminando a necessidade de sistemas separados e garantindo consistência entre processamento histórico e em tempo real.

Baixa Latência

Processamento em milissegundos com throughput de milhões de eventos por segundo

Exactly-Once

Garantias de consistência stateful com checkpointing distribuído

Event Time

Processamento baseado no timestamp real dos eventos, não no tempo de chegada

Stateful Processing

Mantém estado distribuído com recuperação automática de falhas

Casos de Sucesso

Alibaba

1T+

eventos/dia em busca e recomendações



Uber

470M+

transações/segundo em picos



Netflix

5B+

eventos/dia em análise de sessões



Booking.com

10B+

eventos/dia em personalização



Versão Atual: Flink 2.2

Lançado em Dezembro 2025 com recursos de IA avançados

Arquitetura Interna: JobManager e TaskManager

Componentes Principais



Dispatcher

Ponto de entrada para submissão de jobs via Web UI, CLI ou REST API. Encaminha jobs para o JobManager.



JobManager (Coordenador)

O "cérebro" do Flink. Recebe jobs, quebra em tarefas físicas, cria plano de execução, monitora progresso e gerencia checkpoints.



ResourceManager

Gerencia alocação de recursos e slots. Comunica-se com Kubernetes ou YARN para provisionar recursos adicionais.



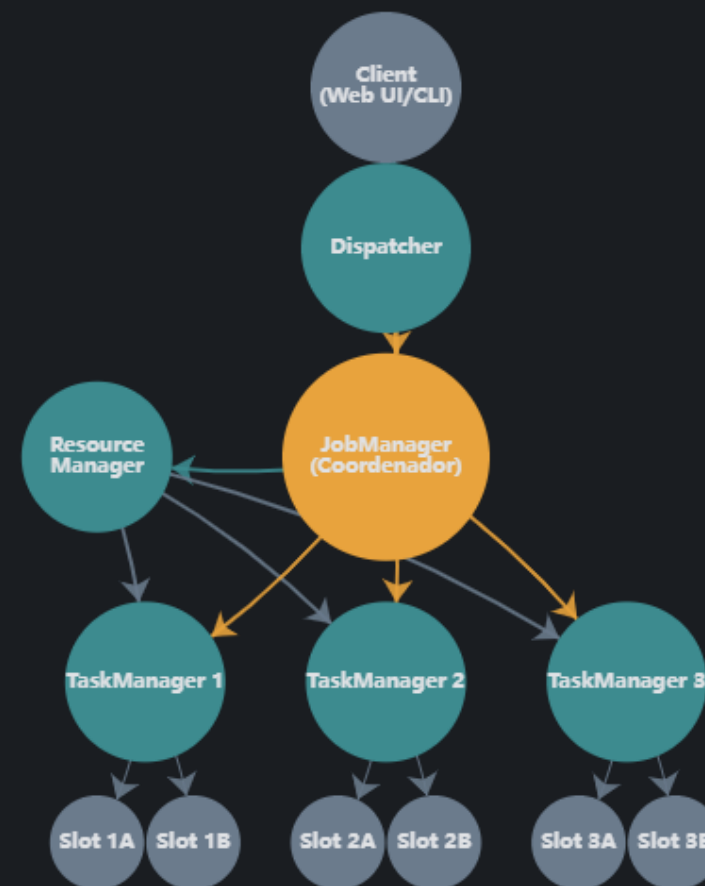
TaskManagers (Workers)

Executam tarefas de computação. Cada TaskManager possui múltiplos slots para execução paralela de tarefas.

Fluxo de Execução de um Job

- 1 **Submissão:** Usuário envia job para Dispatcher
- 2 **Planejamento:** JobManager cria JobGraph
- 3 **Alocação:** ResourceManager aloca slots
- 4 **Execução:** TaskManagers processam em paralelo
- 5 **Checkpointing:** Snapshots periódicos para tolerância
- 6 **Monitoramento:** Dashboard e métricas em tempo real

Arquitetura do Cluster Flink



02 Slots, Paralelismo e Execução Distribuída

Task Slots: Unidade de Recursos

Cada **TaskManager** é dividido em **Task Slots**, que representam a unidade básica de alocação de recursos no Flink. Um slot pode executar uma cadeia de tarefas (pipeline) do fluxo de dados.

Configuração típica:

Slots por TM: **4-8** Memória por slot: **1-4 GB**
CPUs por slot: **1-2 cores** Network buffers: **2048+**

Paralelismo e Escalabilidade

O **paralelismo** define quantas instâncias de uma operação são executadas simultaneamente. O Flink distribui tarefas automaticamente entre slots disponíveis.

- 1 **Nível de operação:** Cada operador pode ter paralelismo diferente
- 2 **Nível de job:** Paralelismo padrão para todas as operações
- 3 **Escalabilidade:** Adicione TMs dinamicamente para aumentar throughput

Shuffle e Troca de Dados

O **shuffle** é o processo de redistribuição de dados entre tarefas paralelas. O Flink otimiza automaticamente o shuffle para operações como `keyBy()`, `broadcast` e `rebalance`.

Distribuição de Tarefas em Slots

Slot-1	T4	T8	T12	T16
Slot-2	T3	T7	T11	T15
Slot-3	T2	T6	T10	T14
Slot-4	T1	T5	T9	T13
	TM-1	TM-2	TM-3	TM-4

Memory Management

Slots isolam memória entre tarefas, prevenindo interferência

Network Stack

Netty-based com buffers otimizados para throughput

Checkpointing: Tolerância a Falhas e Exactly-Once

Mecanismo de Checkpointing

O **checkpointing** é o mecanismo de snapshot distribuído do Flink que garante **tolerância a falhas**. Barreiras de checkpoint (elementos especiais) são injetadas periodicamente no fluxo de dados.

Exactly-Once (Padrão)

Garante que cada registro afete o estado exatamente uma vez, mesmo com falhas

At-Least-Once

Maior throughput, mas possível duplicação de registros em falhas

State Backends



HashMapStateBackend

Estado em memória (heap). Rápido, mas limitado pela RAM. Ideal para estados pequenos.



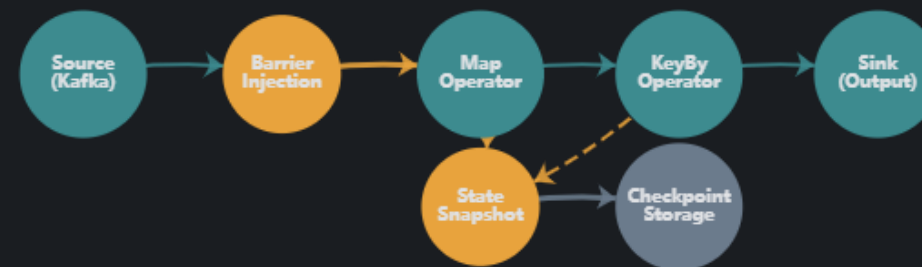
EmbeddedRocksDBStateBackend

Estado em disco (LSM-tree). Suporta estados maiores que a memória. Checkpointing incremental.

Configurações Principais

Intervalo:	1-10 min	Timeout:	10 min
Min pause:	500ms	Max concurrent:	1

Fluxo de Checkpointing



Recuperação de Falhas

- 1 JobManager detecta falha no TaskManager
- 2 Cancela tarefas afetadas
- 3 Restaura estado do último checkpoint
- 4 Reinicia tarefas a partir do checkpoint
- 5 Continua processamento sem perda de dados

Integrações Open Source: Kafka, Kubernetes e Ecossistema



Apache Kafka

Principal fonte e sink de streams. O Flink consome de topics Kafka com **exactly-once semantics** e publica resultados de volta.

Kafka Source: Consumo paralelo com controle de offsets

Kafka Sink: Escrita transacional com 2-phase commit

Schema Registry: Integração Avro/Protobuf/JSON Schema



Kubernetes

Orquestração nativa de clusters Flink com **auto-scaling**, gerenciamento de recursos e deployment declarativo.

Flink Kubernetes Operator: CRDs para aplicações Flink

Native Integration: Kubernetes como ResourceManager



Bancos de Dados

- ✓ **JDBC:** PostgreSQL, MySQL, Oracle
- ✓ **NoSQL:** MongoDB, Cassandra, Redis
- ✓ **Search:** Elasticsearch
- ✓ **Columnar:** ClickHouse, Apache Druid
- ✓ **Data Warehouses:** Snowflake, BigQuery



Formatos de Dados

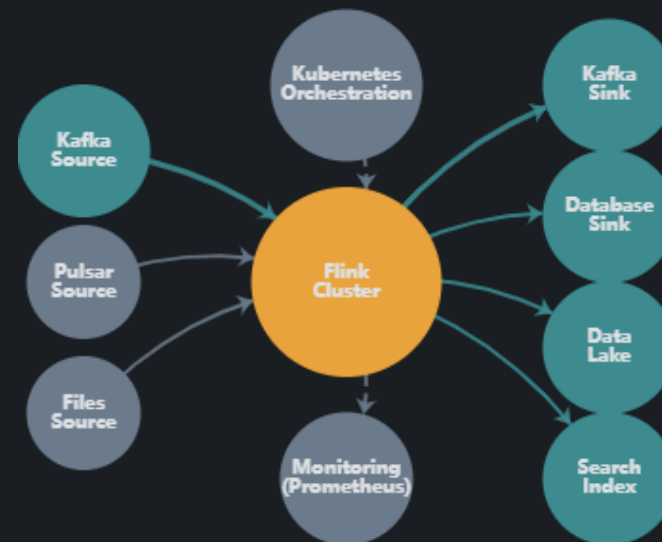
- | | |
|---------|----------|
| Parquet | Avro |
| ORC | JSON |
| CSV | Protobuf |



Cloud Storage

- Amazon S3
- Azure Blob Storage
- Google Cloud Storage

Arquitetura de Integração



05 Camada de IA: ML_PREDICT e Inferência em Tempo Real

Flink 2.1/2.2: A Era da IA

O Flink evoluiu de um motor de processamento de dados para uma **plataforma unificada Data + AI**, permitindo inferência de modelos ML diretamente em pipelines de streaming.

ML_PREDICT (Flink 2.1+)

Função TVF para inferência de modelos em SQL. Suporta OpenAI e providers customizados.

VECTOR_SEARCH (Flink 2.2+)

Busca por similaridade de vetores em tempo real para RAG e embeddings.

Model DDL & Table API

Definição de modelos via SQL DDL ou programaticamente em Java/Python.

Exemplo: ML_PREDICT em SQL

```
-- Criar modelo LLM
CREATE MODEL sentiment_model
INPUT (text STRING)
OUTPUT (sentiment STRING)
WITH (
  'provider' = 'openai',
  'model' = 'gpt-4.1',
  'system-prompt' = 'Classify sentiment'
);

-- Usar modelo em query
SELECT * FROM ML_PREDICT(
  TABLE customer_reviews,
  MODEL sentiment_model,
  DESCRIPTOR(review_text)
);
```

Integração com Frameworks ML

TensorFlow

Modelos SavedModel para inferência em streaming

PyTorch

TorchScript e modelos ONNX integrados

Scikit-learn

Modelos serializados com PMML/ONNX

XGBoost

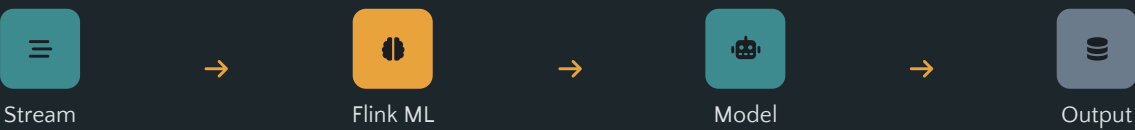
Modelos de árvore para predição em tempo real

Exemplo: Table API (Java)

```
// Criar e usar modelo
tEnv.createModel(
  "my_model",
  ModelDescriptor.forProvider("openai")
    .inputSchema(...)
    .outputSchema(...)
    .option("model", "gpt-4.1")
    .option("async", "true")
    .build()
);

Model model = tEnv.fromModel("my_model");
Table result = model.predict(inputTable, ...);
```

Arquitetura de Inferência



Lakehouse e Formatos de Tabela: Iceberg, Delta, Paimon

Arquitetura Lakehouse

O **lakehouse** combina a escalabilidade e abertura de data lakes com a confiabilidade transacional e governança de data warehouses. O Flink desempenha papel central na **unificação de batch e streaming**.

Apache Iceberg

Formato agnóstico de engine com metadados hierárquicos. Suporta Spark, Flink, Trino, Dremio.

REST Catalog Time Travel

Delta Lake

Log transacional append-only. Otimizado para Spark/Databricks. UniForm para leitura Iceberg.

Z-Ordering Liquid Clustering

Apache Hudi

Otimizado para CDC e upserts. Modos Copy-on-Write e Merge-on-Read.

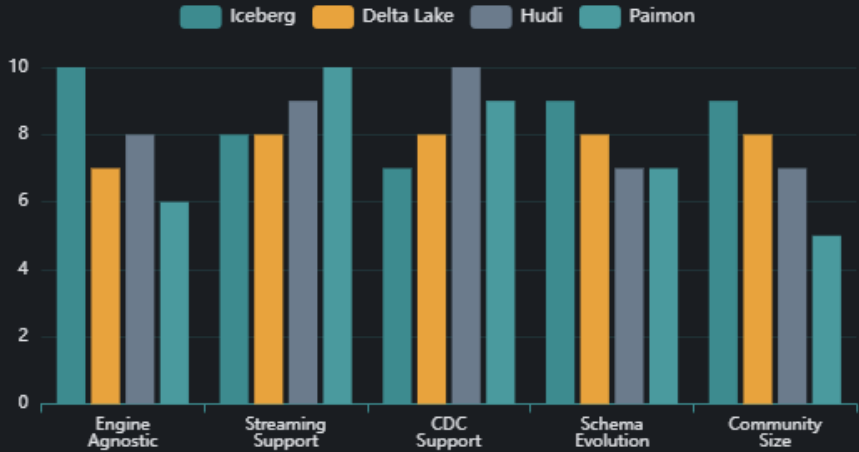
CoW MoR

Apache Paimon

Streaming-first lakehouse com LSM-tree. Integração nativa com Flink.

CDC Compaction

Comparação de Formatos



Flink no Lakehouse

- ✓ **Streaming Ingestion:** Flink escreve streams diretamente em tabelas Iceberg/Delta/Hudi
- ✓ **Batch Processing:** Mesmo código processa dados históricos
- ✓ **Unified Analytics:** Query engines leem dados processados pelo Flink
- ✓ **CDC Integration:** Captura mudanças de bancos para lakehouse em tempo real

Exemplo: Flink + Iceberg

```
-- Criar catálogo Iceberg
CREATE CATALOG iceberg_catalog WITH (
  'type' = 'iceberg',
  'catalog-type' = 'rest',
  'uri' = 'http://polaris:8181'
);

-- Inserir stream em tabela Iceberg
INSERT INTO iceberg_catalog.db.events
SELECT * FROM kafka_source;
```

Tecnologias Emergentes em Data Infrastructure 2025-2026



AI-Native Development

Plataformas de desenvolvimento com **IA generativa** revolucionam a criação de software. Equipes pequenas constroem aplicações sofisticadas com ferramentas de coding assistido.

Impacto: Ciclos de desenvolvimento 10x mais rápidos

Flink: Geração automática de pipelines SQL



Edge AI Deployment

Colocar workloads de IA fisicamente mais próximos dos usuários para **reduzir latência** e melhorar performance em aplicações real-time.

Drivers: IoT, autonomous vehicles, AR/VR

Flink: Processamento de streams no edge



Energia Sustentável

Data centers consomem **4% da energia global** (2024), previsto dobrar até 2030. Foco em renováveis e reatores nucleares modulares (SMRs).

Soluções: Hidrogênio, SMRs, baterias sódio-íon

Flink: Eficiência energética em processamento



Multi-Agent Systems

Sistemas de **agentes autônomos** colaboram em tarefas complexas, tomando decisões e agindo em nome dos usuários.

Casos: Customer service, supply chain optimization

Flink: Orquestração de eventos entre agentes



Quantum-AI Convergence

Data centers se preparando para a **computação quântica**. Chip do Google em 2025: 13.000x mais rápido que máquinas convencionais.

Preparação: Infraestrutura quantum-ready

Flink: Integração futura com algoritmos quânticos



Data Strategy Priority

61% dos líderes de TI priorizam estratégia de dados. Aumento da adoção de PostgreSQL e open source.

Adoção MSP: 60% usam provedores de serviços

Flink: Plataforma open source estratégica

Tendências em Números

Adoção Open Source **61%**

Uso de MSPs **60%**

Crescimento Edge AI **85%**

Casos de Uso: Detecção de Anomalias, IoT e Analytics



IoT Sensor Processing

Planta industrial com 10.000 sensores monitorando temperatura, pressão e vibração. Detecção de anomalias 15-30 minutos antes de falhas.

```
// Janela de 1 minuto com watermark
WatermarkStrategy.
  forBoundedOutOfOrderness(Duration.ofSeconds(30))
```



Clickstream Analytics

Empresa de mídia analisa páginas em tendência, navegação de usuários e abandono de sessões em tempo real para decisões editoriais.

```
-- Janela deslizante de 15 min
SELECT article_id,
  COUNT(DISTINCT user_id) readers
FROM page_views
GROUP BY HOP(event_time, INTERVAL '1' MINUTE, INTERVAL '15' MINUTE)
```



Customer 360

Varejista mantém perfis de clientes combinando e-commerce, lojas físicas, programa de fidelidade e engajamento de email em tempo real.

```
-- Join temporal de streams
SELECT o.customer_id, o.order_total,
  c.loyalty_tier, c.lifetime_value + o.order_total
FROM order_stream o
JOIN customer_stream c
  ON o.customer_id = c.customer_id
```



Notification Engines

SaaS envia notificações em tempo real com deduplicação e rate limiting usando estado keyed por user_id.

- ✓ Exactly-once delivery garantido
- ✓ Rate limiting por usuário
- ✓ Deduplicação com estado

Outros Casos de Uso

🛡️ Fraud Detection

💰 Dynamic Pricing

📈 Real-time ETL

🔧 Network Monitoring

Ecossistema Flink: Ferramentas Complementares



Ververica Platform

Plataforma empresarial para gerenciamento de clusters Flink com **auto-scaling**, SQL editor, e governança.

- ✓ Ververica SQL
- ✓ Universal Streaming
- ✓ Gerenciamento de estado



Apache Paimon

Lakehouse **streaming-first** com LSM-tree. Integração nativa com Flink para CDC e queries incrementais.

- ✓ Snapshot isolation
- ✓ Continuous compaction
- ✓ CDC ingestion



Streaming Databases

RisingWave

Materialized views contínuas sobre streams com saída para Iceberg

Materialize

Views materializadas SQL em tempo real com consistência strict serializable

ksqlDB

SQL nativo Kafka para transformações streaming



Monitoramento

 **Prometheus:** Métricas e alertas

 **Grafana:** Dashboards visuais

 **Jaeger:** Distributed tracing



Orquestração

Apache Airflow

Orquestração de pipelines batch e streaming

Dagster

Orquestração data-aware com lineage

Prefect

Workflow orchestration moderno

Ecossistema Completo



Flink



Kafka



K8s



Iceberg



Grafana



AI/ML

Comunidade Apache

Flink é um projeto **top-level Apache** com 116+ contribuidores globais, 16 FLIPs no release 2.1, e 220+ issues resolvidas.

Apache Flink: A Base da Infraestrutura de Dados Moderna

O Apache Flink representa o estado da arte em processamento de streams, unificando batch e streaming com garantias exactly-once, integrações ricas com ecossistema open source, e capacidades emergentes de IA.

1T+

Eventos/dia na Alibaba

2.2

Versão atual com IA

116+

Contribuidores globais

**Real-Time**

Sub-segundo latência

**AI-Ready**

ML_PREDICT integrado

**Open Source**

Apache Software Foundation